Livrable E5

Résolution d'incident

Franck Le Fur

Le 30 Mars 2026

Titre professionnel **Développeur en intelligence artificielle**
de niveau 6 enregistré au RNCP sous le n°37827.

Passage par la voie de la formation - parcours de 19 mois achevé en mars 2026

Table des matières

1 Présentation **2**
1.1 Instructions du projet 2
1.2 Spécifications techniques 2

2 Découverte de l'application **2**
2.1 Etude de l'application 2
2.2 Création de l'environnement 3

3 Debug de l'application [C21] **3**
3.1 Recherche du bug de l'application 3
3.2 Comprendre l'erreur 4
3.3 Correction du bug 4

4 Mise en place de sécurité [C21] **4**
4.1 Mise en place d'un système de logging 4
4.2 Mise en place de tests unitaires 5
4.2.1 Test de chargement d'image 5
4.2.2 Test de chargement du modèle Keras 5
4.2.3 Test de compatibilité des dimensions 6
4.2.4 Test sur les probabilités de la prédiction 6
4.3 Dépoiement de tests unitaires avec GitHub Action 6
4.4 Mise en place d'un système d'alerting 6
4.5 Mise en place d'un système de monitoring 7

5 Boucle MLOPS [C20] **7**
5.1 Concept MLOPS 7
5.2 Modification du formulaire HTML 8
5.3 Enregistrement des feedback en base de données 8
5.4 Boucle de réentrainement 8

6 Conclusion **8**

7 Annexes **9**
7.1 Résultats de l'application 9
7.2 GitHub Action 9
7.3 Flask Monitoring Dashboard 10
7.4 Séparation des données d'entraînement 10

1 Présentation

1.1 Instructions du projet

A partir du code d'une application, résoudre les incidents techniques en apportant les modifications nécessaires, en document les solutions pour en garantir le fonctionnement opérationnel.

1.2 Spécifications techniques

Ce projet sera intégralement mené en Python, ce langage offre gratuitement toutes les librairies nécessaires à la conduite de ce projet. Le présent rapport est rédigé en Latex via l'utilitaire en ligne overleaf. Enfin le dépôt GitHub du projet est consultable ici :
https://github.com/Franck-LF/livrable_BUG
Une application **Flask** de classification d'images satellites a été fournie, elle contient un bug, nous devons analyser, rechercher le bug, débugger l'application.

Pour la phase de debug nous utiliserons la **méthodologie** suivante :

- Comprendre la structure de l'application (architecture et fichiers de code),
- Reproduire le problème
- Faire des hypothèses et les tester
- Corriger le code et faire des tests
- Documenter les corrections

Dans un second temps, le debug devra être accompagné de la mise en place de systèmes de sécurité suivants :

- un ou plusieurs tests unitaires pour éviter que le bug ne réapparaisse,
- un système de logging dans l'application pour pouvoir monitorer l'application,
- un système d'alerting pour envoyer automatiquement des emails lors d'une erreur,

Enfin, dans une **démarche MLOps** d'amélioration continue des modèles d'IA, nous mettrons en place un système de **feedback loop** qui permet de demander à l'utilisateur son avis sur la prédiction faite puis enregistre en base de données cet avis avec les données, ces données "augmentées" pourront servir plus tard pour un réentrainement du modèle.

2 Découverte de l'application

2.1 Etude de l'application

Avant de commencer à lire le code de l'application, on s'intéresse à l'architecture de l'application, les fichiers, L'application est composée de fichiers suivant l'architecture suivante :

- Projet
  - app.py
  - README.md
  - requirements.txt
  - templates
    - feedback_ok.html
    - result.html
    - upload.html
  - models
    - final_cnn.keras
  - image_to_test

Le fichier `app.py` contient l'application principale (application Flask). Le fichier `requirements.txt` contient la liste des modules python à installer pour lancer notre application. Le dossier **models** contient le model **Keras** déjà entraîné. Le dossier **templates** contient les pages HTML de l'application.

Le sous-dossier **templates** contient les pages html de l'application,

- **upload.html** :
- **result.html** :
- **feedback_ok.html** : Page affichant que le feedback a bien été envoyé.

2.2 Création de l'environnement

Pour pouvoir lancer l'application on crée un environnement python puis on installe les librairies python nécessaires.

```
conda create -n livrable_BUG
pip install -r requirements.txt
```

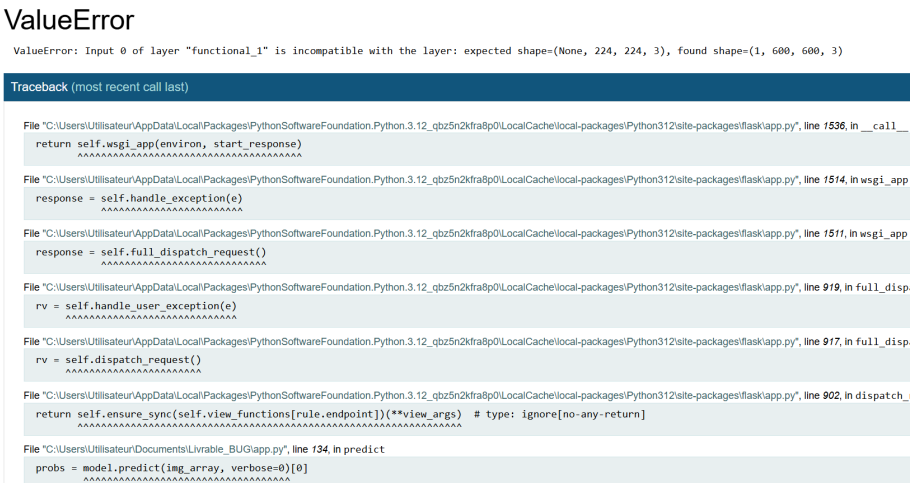
Ensuite on peut lancer l'application avec l'instruction `python app.py`

3 Debug de l'application [C21]

3.1 Recherche du bug de l'application

Méthodologie : On lance l'application avec la commande `python app.py`

L'application **Flask** se lance via le navigateur, nous arrivons sur la page d'accueil nous invitant à choisir un fichier, nous choisissons un fichier image et demandons à l'application de faire une prédiction, à savoir s'il s'agit d'une photo de désert, de montagne ou d'une prairie.
On obtient le message d'erreur suivant :



3.2 Comprendre l'erreur

Le message principale de l'erreur indique clairement une incompatibilité entre deux tailles de tenseurs.

ValueError

```
ValueError: Input 0 of layer "functional_1" is incompatible with the layer: expected shape=(None, 224, 224, 3), found shape=(1, 600, 600, 3)
```

Comme nous travaillons avec un modèle **keras**, nous pouvons soupçonner que la taille de l'objet donné en entrée de **Keras** ne correspond pas à la taille d'entrée de notre modèle **Keras**

La suite de l'erreur confirme notre intuition en indiquant la ligne d'erreur où le modèle tente de faire sa prédiction.

File "C:\Users\Utilisateur\Documents\Livrable_BUG\app.py", line *134*, in predict

```
probs = model.predict(img_array, verbose=0)[0]
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

3.3 Correction du bug

Nous allons ajouter du code pour redimensionner l'image aux dimensions du tenseur d'entrée du modèle **Keras**.

1ère étape :
Dans la fonction `predict`, nous récupérons la dimension du tenseur d'entrée du modèle $(None, H, W, 3)$, et nous transmettons les composantes (H, W) à la fonction qui traite l'image.

```
# Recuperation de la dimension des tenseurs d'entree du modele keras
input_shape = model.get_config()["layers"][0]["config"]["batch_shape"]
print("Input_shape", input_shape)
# On ajoute cette dimension en argument de la fonction qui fait le traitement de l'
    image
img_array = preprocess_from_pil(pil_img, target_size = input_shape[1:3])
```

2ème étape :
Nous transmettons cette dimension à la fonction qui traite l'image, pour cela nous ajoutons un argument supplémentaire.

```
def preprocess_from_pil(pil_img: Image.Image, target_size:tuple) -> np.ndarray:
    img = pil_img.convert("RGB")
    # Redimensionnement de l'image
    img = img.resize(target_size)
    img_array = np.asarray(img, dtype=np.float32)
    img_array = np.expand_dims(img_array, axis=0)
    return img_array
```

Désormais toutes les images seront redimensionnées pour correspondre aux dimensions du tenseur d'entrée du modèle.
La prédiction s'effectue correctement, le bug est corrigé.
Voir Annexe - Resultat de l'application

4 Mise en place de sécurité [C21]

4.1 Mise en place d'un système de logging

On utilise la librairie **Logging** pour tracer les informations importantes lors de l'utilisation de l'application.

1er Exemple :
Message de log **warning** si le fichier saisi par l'utilisateur n'est pas conforme.

```
if file.filename == "":
  logger.warning("Aucun fichier selectionne pour l'upload.")
else:
  logger.warning(f"Extension de fichier non autorisee: {file.filename}")
```

2ème Exemple :
Si un fichier image ne peut être lu par notre librairie PILLOW, on affiche un message d'erreur dans les logs puis on renvoie vers la page d'accueil.

```
try:
  pil_img = Image.open(io.BytesIO(raw))
except:
  logger.error(f"Erreur lors de l'ouverture de l'image : {file.filename}")
  return redirect("/")
```

3ème Exemple :
On affiche un message informatif indiquant la dimension initiale de l'image.
Après redimensionnement, si l'image convertie en tableau numpy n'a pas les dimensions des tenseurs d'entrée du modèle Keras, on affiche un message d'erreur dans les logs et on renvoie vers la page d'accueil.

```
  # On affiche la dimension initiale de l'image
  logger.info(f"taille initiale de l'image : {img.size}")
  if img_array.shape[1:] != input_shape[1:]:
    logger.error(f"Apres traitement de l'image, la taille de l'image incorrecte: {
   img_array.shape[1:]} au lieu de {input_shape[1:]}")
    return redirect("/")
```

```
2025-09-17 09:22:29,210 Fichier choisi : desert_96.jpg
2025-09-17 09:22:29,302 taille initiale de l'image : (600, 600)
```

4.2 Mise en place de tests unitaires

Nous utilisons la librairie PyTest pour mettre en place des tests unitaires.
Nous allons faire un déploiement de notre application sur GitHub et activer les tests unitaires via le service GitHub Action.

Pour cela on crée un script `test_app.py` contenant tests unitaires.

4.2.1 Test de chargement d'image

On crée un test pour vérifier que les images se chargent correctement.

```
  def test_load_image():
    img_path = "images_to_test/desert_96.jpg"
    try:
      _ = Image.open(img_path).convert("RGB")
      print(f"Image chargee a partir de {img_path}")
    except:
      assert False, f"Impossible de charger l'image depuis {img\_path}."
```

4.2.2 Test de chargement du modèle Keras

On crée un test pour vérifier que le modèle Keras se charge correctement.

```
  def test_load_model():
    model\_path = "models/final_cnn.keras"
    try:
        _ = keras.saving.load_model(model_path, compile=False)
    except:
        assert False, f"Impossible de charger le modele depuis {model_path}."
```

4.2.3 Test de compatibilité des dimensions

On crée un test pour vérifier que les les dimensions du tableau numpy (conversion de notre image) correspond aux dimensions du tenseur d'entrée du modèle.

```
def test_image_input_model():
  img_path = PATH + "images_to_test/meadow_89.jpg"
  model_path = PATH + "models/final_cnn.keras"
  img = Image.open(img_path).convert("RGB")
  model = keras.saving.load_model(model_path, compile=False)
  input_shape = model.get_config()["layers"][0]["config"]["batch_shape"]
  img_array = preprocess_from_pil(img, target_size = input_shape[1:3])
  assert img_array.shape[1:] == input_shape[1:], \
    f"ATTENTION !!! La taille de l'image ne correspond pas a la taille 'entree du modele\
    \n {img_array.shape[1:]} != {input_shape}"
```

4.2.4 Test sur les probabilités de la prédiction

On crée un test pour vérifier que les valeurs retournées par le modèle lors de la prédiction sont bien comprises entre 0 et 1.

```
def test_probability():
  img_path = PATH + "images_to_test/meadow_89.jpg"
  model_path = PATH + "models/final_cnn.keras"
  img = Image.open(img_path).convert("RGB")
  model = keras.saving.load_model(model_path, compile=False)
  input_shape = model.get_config()["layers"][0]["config"]["batch_shape"]
  img_array = preprocess_from_pil(img, target_size = input_shape[1:3])
  probas = model.predict(img_array, verbose=0)
  print("proba:", probas)
  assert all([p >= 0 and p <= 1 for p in item] for item in probas), "Une probabilite n'est pas comprise entre 0 et 1"
```

4.3 Dépoiement de tests unitaires avec GitHub Action

Nous ajoutons un fichier **main.yml** dans le répertoire **.github/workflows**, ce fichier contient les instructions que GitHub Action exécutera lors de chaque `git push`.
Nous y ajoutons l'exécution de pytest qui va systématiquement exécuter toutes les fonctions ayant le préfixe "test_" de tous les fichiers ayant le préfix "test_".

```
- name: Run tests
  run: KERAS_BACKEND=torch pytest -vv
```

Nous pouvons suivre sur Github le déroulement des actions.
Voir Annexe

4.4 Mise en place d'un système d'alerting

Les développeurs n'analysent pas systématiquement les logs, ceux-ci servent surtout lorsqu'une panne apparait permettant ainsi de trouver la source de l'erreur. Le système d'**alerting** envoie des emails lorsqu'une erreur critique intervient dans l'exécution de l'application, c'est une démarche de prévention, on peut intervenir sur l'application avant que les utilisateurs ne signalent un disfonctionnement.

Nous utilisons la librairie **Logging** pour faire de l'alerting.
Nous créeons un objet SMTPHandler capable d'utiliser le protocole SMTP et donc d'envoyer des emails, ensuite nous précisons que seuls les messages d'erreur devront être transmis par emails.

```
mail_handler = SMTPHandler(
  mailhost=("smtp.gmail.com", 587),
  fromaddr="f7lefur@gmail.com",
  toaddrs=["f7lefur@gmail.com"],
  subject="Alerte : une erreur est survenue",
  credentials=(os.getenv('GMAIL_USER'), os.getenv('PASSWORD')),
  secure=()
)
logger.setLevel(logging.ERROR)  # Seuls les logs >= ERROR seront envoyes
logger.addHandler(mail_handler)
```

Désormais, chaque message de log de type
`logger.error("Erreur ...", exc_info=True)`
enverra un email pour avertir d'une erreur.

4.5 Mise en place d'un système de monitoring

On installe la librairie Flask_Monitoring Dashboard qui permet une analyse détaillée d'une application **Flask**.

Flask-MonitoringDashboard permet de :

- compter le nombre de requêtes par endpoint, utilisateur ou période,
- mesurer le temps de réponse des endpoint,
- afficher l'origine des requêtes,
- identifier les routes lentes ou les requêtes problématiques.

```
import flask_monitoringdashboard as dashboard
# On relie notre application Flask au dashboard
dashboard.bind(app)
```

Dès l'exécution de l'application Flask, cela va créer un fichier .db à la racine du projet, c'est un fichier de base de données Flask-Monitoring.

Nous ajoutons à la racine de notre projet un fichier `config.cfg` contenant les paramètres de notre outil de monitoring.

On précise l'emplacement du fichier de base de données :

`DATABASE=sqlite:///C:/Users/Utilisateur/Documents/Livrable_BUG/dashboard.db`

La base de données par défaut de l'outil de monitoring est **SQLite**.

Ensuite on accède au dashboard via le navigateur en local `http://localhost:5000/dashboard/`
Voir Annexe

5 Boucle MLOPS [C20]

5.1 Concept MLOPS

Une démarche **MLOPS** consiste en l'amélioration continue d'un modèle, c'est-à-dire qu'après avoir entrainé une première fois un modèle nous réalisons en boucle les étapes suivantes :

- modification de la base de données d'entrainement (généralement par augmentation de donées),
- réentrainement du modèle sur la nouvelle base de données,
- évaluation du "nouveau" modèle.
- dans le cas d'une amélioration : déploiement du nouveau modèle

Pour se faire, il faut permettre la modification des données d'entrainement.

5.2 Modification du formulaire HTML

Nous allons proposer aux utilisateur d'envoyer un **feedback** concernant les prédictions faites par notre modèle. Cela nous permettra d'enrichir / corriger notre base de données pour un futur réentrainement.

Nous ajoutons au code HTML des boutons radios correspondants aux 4 catégories d'images, l'utilisateur peut alors envoyer son feedback.

Selon vous, quelle est la bonne catégorie ?

○ Desert ○ Forest ○ Meadow ○ Mountain [Envoyer un feedback]

5.3 Enregistrement des feedback en base de données

Dans l'application **Flask** nous créons un objet vers notre base de données **MongDB** puis un objet vers notre collection de feedbacks.

```
client = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = client["satellite"]
col_feedbacks = mydb["feedback"]
```

Ensuite, à chaque fois qu'un utilisateur enverra un feedback nous enregistrerons en base les éléments suivants :

- L'image au format base64,
- la prédiction de notre modèle,
- le score de confiance du modèle sur sa prédiction,
- le feedback utilisateur (parmi les 4 catégories).

```
my_dict = {  'image' : image_data_url,
             'predition' : predicted_label,
             'score' : confidence,
             'feedback' : feedback  }
col_feedbacks.insert_one(my_dict)
```

Les feedback s'enregistrent bien dans notre base **MongoDB**.

```
_id: ObjectId('68ecbf12446f95f03dcc59fe')
image : "…"
predition : "meadow"
score : "0.9749481678009033"
feedback : "desert"
```

5.4 Boucle de réentrainement

Plusieurs stratégies de séparation des données sont alors possibles pour le réentrainement, le point fondamental étant de ne jamais utiliser les données d'entrainement pour faire les tests.

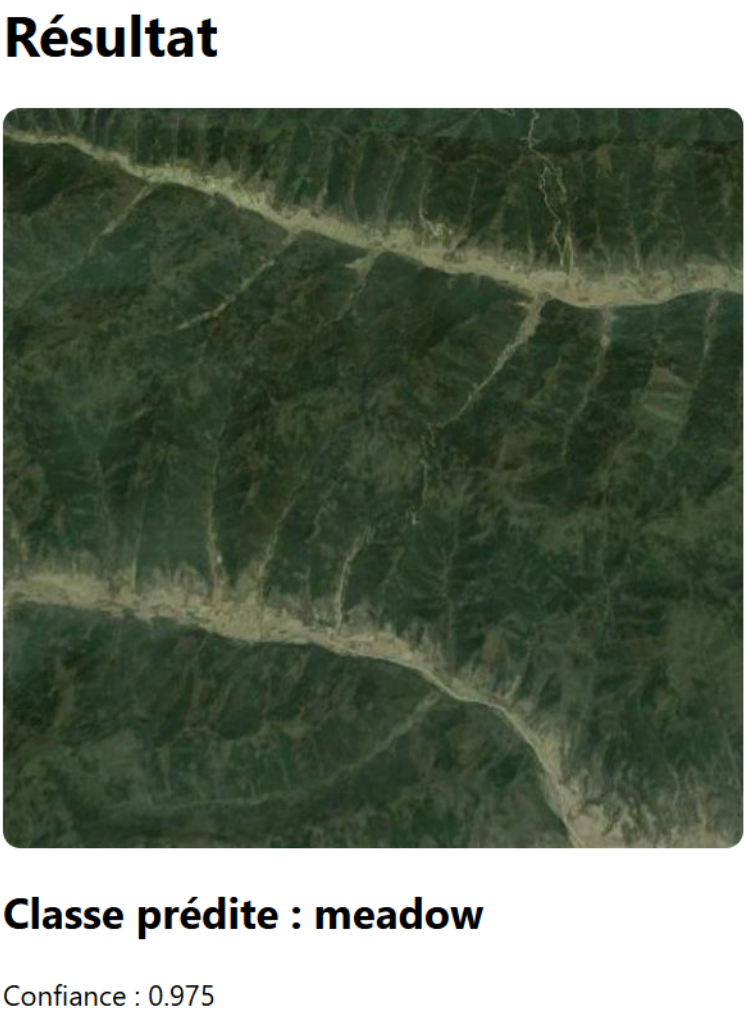
Nous choisissons de diviser nos données en données d'entraînement et en données de tests tout en conservant le jeu de test initial, cela permet de contrôler une éventuelle régression si une regression de notre modèle est constatée. Voir Annexe - Séparation des données

6 Conclusion

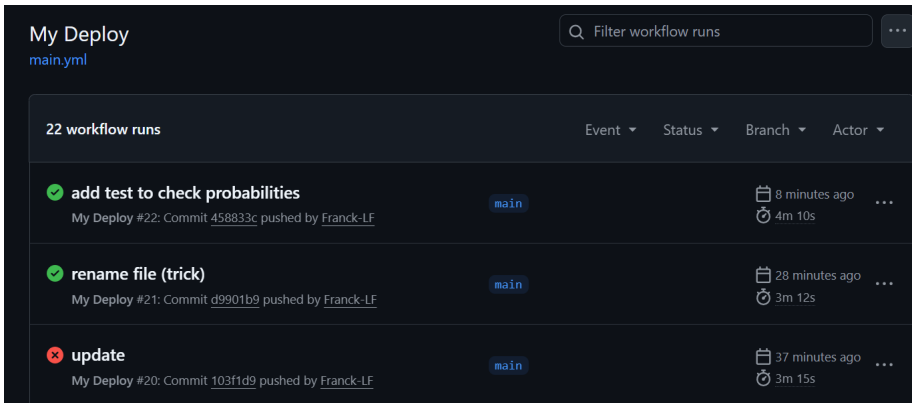
Ce projet a permis de travailler sur une méthodologie de deboguage d'une application et de mettre en place des systèmes de sécurité pour éviter de nouveaux bogues ou bien pour permettre leur détection plus rapidement grâce à de l'alerting et du logging, enfin une boucle MLOps a été mise en place.

7 Annexes

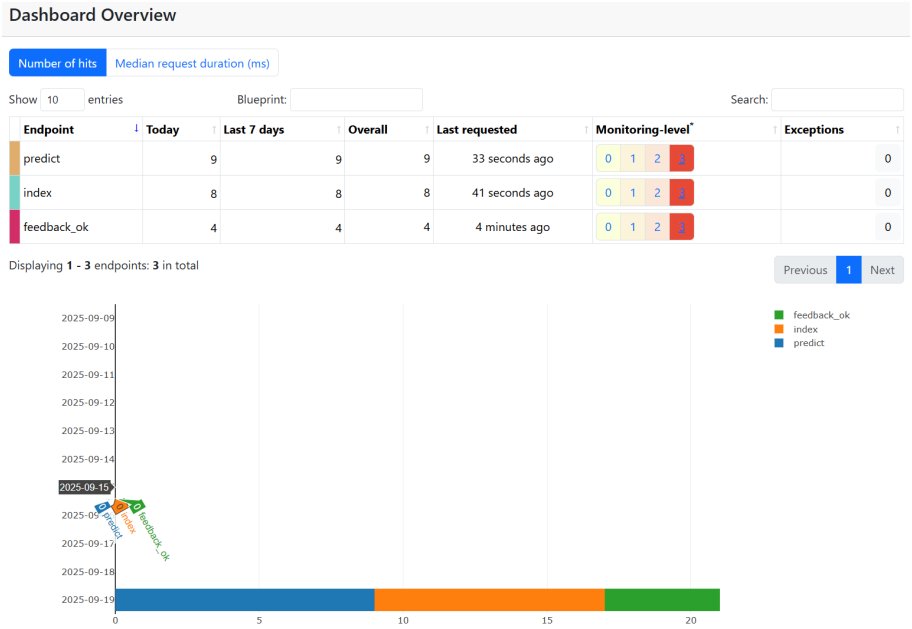
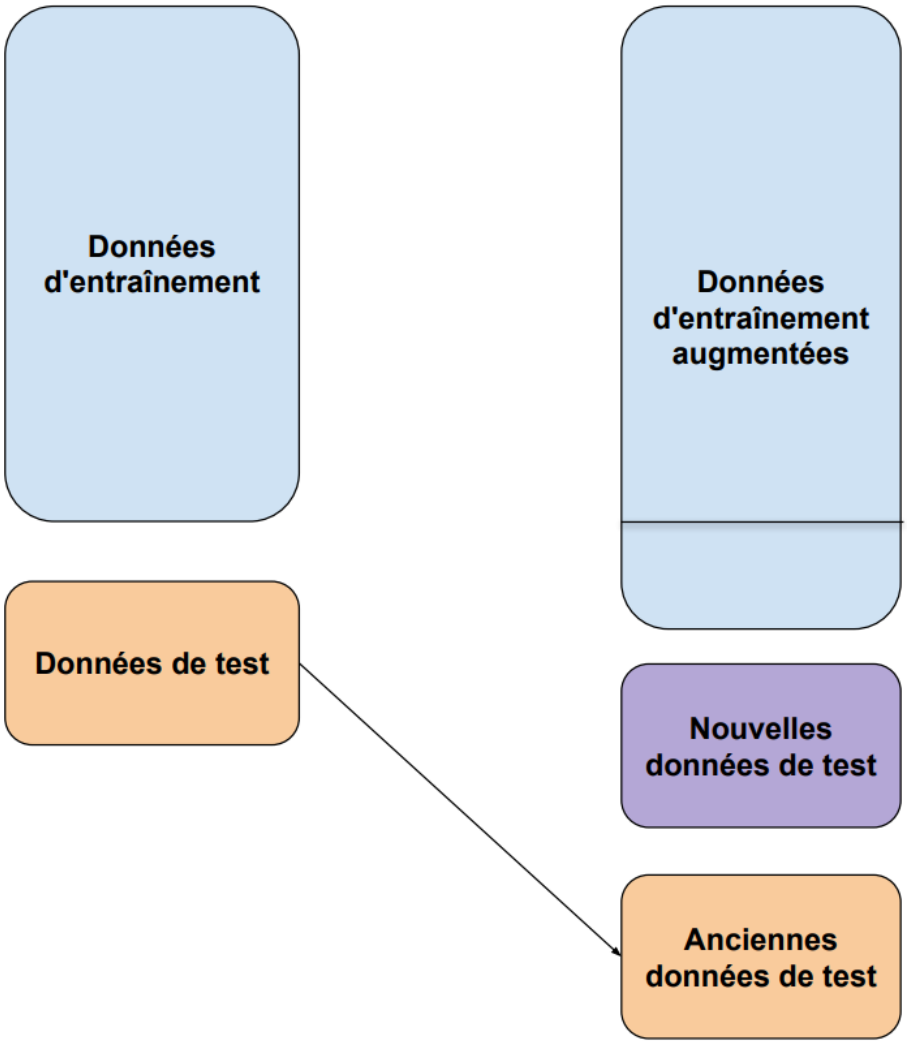
7.1 Résultats de l'application



7.2 GitHub Action



7.3 Flask Monitoring Dashboard



7.4 Séparation des données d'entraînement